

Causal Debugging of Deep Learning Training Failures

Jacques-Charles Gad Senouvo
LambdaSection
github.com/LambdaSection/NeuralDBG

October 3, 2026
Draft v5 — arXiv submission candidate

Abstract

Training deep neural networks fails silently: vanishing gradients, exploding gradients, dead neurons, and data corruption can propagate undetected for hundreds of steps before surfacing as NaN losses. Existing tools monitor metrics but do not trace failures to their root causes. We present NeuralDBG, a causal diagnostic engine that instruments PyTorch’s autograd to extract semantic events and construct directed causal chains linking root causes to final symptoms. Across a combinatorial sweep of 200 architectures spanning 6 families (MLP, CNN, RNN, Transformer, Hybrid, and Black-Swan), NeuralDBG detects 99.4% of injected failures (1,193/1,200) with near-zero false positives on healthy baselines. On out-of-sample validation using four production-grade architectures (torchvision ResNet-18, ViT-Tiny, EfficientNet-B0, Mamba-Mini), it achieves 24/24 detection (100%) with zero false positives (healthy baselines: 1/2/0/0 events). On novel black-swan architectures, Tier 1 (graph neural networks, mixture of experts, diffusion models) reaches 96% detection (104/108) and Tier 2 (FlashAttention, Neural ODE, quantized) reaches 94% (102/108); Tier 4 reaches 100% on RAG with 50% on federated learning (reinforcement learning is covered by a dedicated detector scoring 5/5 on policy-gradient scenarios). We introduce NeuralPrune, a companion diagnostic that identifies redundant parameters without modifying the model, and demonstrate a closed-loop auto-fix pipeline that diagnoses and remediates training failures. A linear probe on frozen Qwen2-0.5B embeddings scores 29/29 on a held-out diagnostic set. The engine, benchmark suite, and interactive notebook are open-source under the MIT license.

1 Introduction

1.1 The Silent Failure Problem

Consider a researcher training a Transformer on a new dataset. At step 3,247, the loss becomes NaN. The researcher checks: learning rate? Data pipeline? Gradient clipping? Each hypothesis takes 15–30 minutes to test. Multiply this by the number of failures per project, and the cost is measured in days.

Existing tools exacerbate this problem by presenting raw metrics without causal interpretation. TensorBoard shows a loss curve spiking—but *why* did it spike? W&B logs gradient norms—but *which layer* caused the explosion? These tools transform the problem from “the loss is NaN” to “here are 50 charts, one of which might explain the NaN.”

1.2 Our Approach

NeuralDBG inverts the debugging workflow. Instead of the user forming hypotheses and checking dashboards, the system:

1. **Hooks** into PyTorch’s autograd to capture forward/backward dynamics
2. **Extracts** semantic events (gradient health transitions, activation regime shifts, optimizer instability)
3. **Builds** a directed causal graph from events
4. **Extracts** ranked causal chains via depth-first search
5. **Diagnoses** root causes and proposes fixes

The key insight: training dynamics form a causal structure. A data anomaly causes a gradient explosion, which causes optimizer divergence. These causal relationships are recoverable from the temporal and layer-wise structure of autograd events.

2 Related Work

NeuralDBG sits at the intersection of three research areas: ML debugging tools, causal inference for software systems, and automated fault localization.

2.1 ML Debugging and Monitoring

The most widely deployed debugging tool is PyTorch’s built-in anomaly detection (`detect_anomaly()` in `torch.autograd`), which traces NaN gradients to specific operations via autograd metadata [1]. While effective for NaN localization, it provides no causal context—it identifies *where* NaN appeared, not *why*. TensorBoard [2] and Weights & Biases [3] log training metrics and support threshold-based alerts, but leave root cause analysis to the practitioner. Amazon SageMaker Debugger [4] captures tensor statistics during training and supports user-defined rules, but rule authoring requires domain expertise and does not generalize across architectures.

Academic debugging tools have explored specific failure modes. DeepXplore [5] uses differential testing across multiple DNNs to detect inconsistencies, targeting correctness bugs rather than training failures. TensorFuzz [6] applies coverage-guided fuzzing to discover numerical issues in TensorFlow graphs. UMLAUT [7] performs static analysis of ML pipeline code to detect configuration errors before execution. Theis et al. [8] propose a statistical framework for detecting anomalous training runs via hypothesis testing on loss curves. None of these tools construct causal explanations linking events across the training timeline.

2.2 Causal Inference and Fault Localization

Statistical fault localization techniques from software engineering—such as Tarantula [9] and Ochiai [10]—rank program statements by their correlation with test failures. These approaches inspired the event-ranking component of NeuralDBG, but program spectra (statement coverage vectors) differ fundamentally from training spectra (gradient and activation time series). Causal discovery algorithms—including PC [11], FCI [12], and Granger causality [13]—infer causal graphs from observational data. NeuralDBG adopts a simpler but more computationally tractable approach:

it constructs causal links from known compatibility patterns rather than learning structure de novo, trading completeness for speed (hook overhead is bounded to a single resource snapshot per training step).

Model interpretability tools such as Captum [14], LIME [15], and SHAP [16] attribute predictions to input features, but attribution is not diagnosis: knowing *which pixels* influenced a prediction does not explain *why training diverged*. Our experiments (§5.12) confirm that Captum’s attribution methods do not identify training failures—they solve a different problem.

2.3 Automated Remediation

The closed-loop auto-fix pipeline described in §5.7 relates to automated program repair (APR) [17] and self-healing systems [18]. Unlike APR, which modifies source code, NeuralDBG’s remediation operates at the training configuration level (learning rate, gradient clipping, optimizer choice), reducing the risk of introducing new bugs. The fine-tuned language model for diagnosis (§5.8) is inspired by recent work on using LLMs for code explanation [19] and bug localization [20], adapted to the domain-specific vocabulary of training dynamics.

2.4 Redundancy Diagnostics

NeuralPrune (§6) addresses a complementary problem: identifying parameters that can be removed without affecting model quality. Existing pruning criteria—magnitude pruning [21], gradient-based pruning [22], and movement pruning [23]—require the user to choose a pruning ratio a priori. NeuralPrune inverts this: it diagnoses *what* is redundant and *how much*, leaving the pruning decision to the practitioner. This is conceptually similar to NetAdapt’s [24] automated channel reduction, but NeuralPrune never modifies the model, acting purely as a diagnostic oracle.

3 Method

3.1 Semantic Event Extraction

NeuralDBG registers forward and backward hooks on every leaf module in a PyTorch model. The forward hook captures:

- Activation statistics (mean, std, sparsity, saturation ratio, dead neuron ratio)
- Data anomalies (NaN, Inf, distribution shifts)
- Hidden state dynamics for RNN modules

The backward hook captures:

- Gradient norms per layer
- Gradient health classification (healthy, vanishing, exploding, saturated)
- Per-gate gradient tracking for LSTM/GRU (input, forget, cell, output gates)
- Trend-based vanishing detection (5-step gradient norm history)

Each hook invocation produces zero or more `SemanticEvent` objects with typed event categories, confidence scores, and structured metadata.

3.2 Causal Graph Construction

Events are filtered to retain only those indicating problematic states (gradient health $\neq$ healthy, activation regime $\neq$ normal, NaN detected, etc.). A causal link is created between two events when:

1. They occur at different layers or are of different types
2. The temporal gap is ≤ 5 steps
3. The event type pair has known causal compatibility (e.g., a data anomaly leading to a gradient transition scores 0.8 confidence)

Compatibility scores are defined in a matrix of 14 type pairs, expanded from 9 in v1.3 to handle RNN-specific event patterns.

3.3 Chain Extraction

Causal chains are extracted via depth-first search through the link graph, with:

- Maximum 30 chains per session
- Minimum 2 links per chain
- Quality gate: at least one genuinely problematic transition
- Node-based cycle detection

Each chain is assigned a root cause (first problematic event) and final symptom (last problematic event), producing readable diagnoses like:

```
data_anomaly[distribution_shift]
  $\to$ gradient_health_transition[exploding]
  $\to$ optimizer_instability[diverging]
```

4 Architecture-Aware Improvements

4.1 The RNN Tuple Problem

A critical limitation was discovered during combinatorial architecture testing. NeuralDBG’s forward hook used `isinstance(output, torch.Tensor)` to gate activation analysis. However, LSTM and GRU modules return tuples (`output_sequence, (h_n, c_n)`) rather than single tensors. This caused ALL activation analysis to be silently skipped for recurrent architectures, resulting in 49% detection for RNNs versus 93% for feed-forward architectures.

Fix: Forward hooks now detect and unwrap RNN output tuples before analysis. Hidden state statistics are captured separately for BPTT gradient health tracking. Combined with mean-pooling over the time dimension and family-adaptive thresholds, RNN detection reaches 100% (198/198) on the canonical sweep.

4.2 Per-Gate Gradient Tracking

LSTM/GRU parameters pack multiple gates into single weight tensors (e.g., `weight_ih_l0` shape `[4*hidden_size, input_size]` for 4 gates). The overall gradient norm can appear healthy while one gate’s gradients vanish. We added post-backward per-gate analysis that splits gradients into per-gate chunks and flags gates where gradient norm $< 5\%$ of max gate norm AND $< 10^{-5}$ absolute. This improved vanishing gradient detection by 20 percentage points (68% $\rightarrow$ 88%).

4.3 Trend-Based Vanishing Detection

Traditional threshold-based vanishing detection (gradient norm $< 10^{-6}$) misses gradual vanishing where the absolute norm remains above threshold but has decreased by $100\times$. We added a 5-step rolling history of gradient norms per layer. If norms consistently decrease and the final norm is $< 20\%$ of the initial norm, a vanishing event is emitted regardless of absolute threshold. This contributed $+10\%$ to vanishing detection.

5 Experimental Validation

5.1 Combinatorial Architecture Sweep

We constructed a combinatorial architecture generator producing configurations across 6 families (200 total):

Table 1: Architecture families and configuration parameters

Family	Configs	Parameters
MLP	50	depths 2–10, widths 16–256, activations, norms, skips
CNN	40	Conv2d, depths 2–5, kernels 3/5, BatchNorm
RNN	40	LSTM/GRU, depths 1–4, bidirectional
Transformer	40	MultiHeadAttention, heads 2–8, d_model 32–128
Hybrid	20	Conv+Linear, Attention+MLP, RNN+MLP
BlackSwan	10	GNN, MoE, Diffusion, RL, RAG, FlashAttn, NeuralODE

Each configuration is tested with a healthy baseline (8 steps) and 6 injected bugs (exploding LR, vanishing gradients, zero init, NaN data, dead bias, divergence).

5.2 Detection Results

Table 2: Detection rates by injected bug (combinatorial sweep, 200 configs $\times$ 6 bugs = 1,200 evaluations, run 2026-08-13)

Bug	Detection
Exploding gradients	100%
Vanishing gradients	100%
Dead bias	100%
Divergence	100%
Zero initialization	98%
NaN data	98.5%
Overall	99.4% (1,193/1,200)

Seven residual misses (1 CNN, 6 FlashAttention) are masked by an absolute saturation heuristic unsuitable for unbounded Linear outputs; fixing it requires an engine change with full re-validation, so we report 99.4% with this limitation disclosed rather than tuning around it.

5.3 Black-Swan Architecture Detection

We extended validation to novel architecture families never tested by any debugging tool. Tier 1 (GNN, MoE, Diffusion) achieved 96% detection (104/108). Tier 2 (FlashAttention, Neural ODE, Quantized) achieved 94% detection (102/108). Tier 4 achieved 100% on RAG and 50% on federated learning; reinforcement learning policy-gradient scenarios are covered by a dedicated detector (5/5). Detailed per-family results are available in the supplementary material.

5.4 Stress Test Suite

We designed 15 stress scenarios targeting extreme training conditions: $10\times$ normal gradients, $0.1\times$ gradients, $10\times$ input scale, NaN/Inf in data, mixed precision (fp16), 100-layer depth, 1K-token attention, LSTM hidden state vanishing, duplicate input consistency, gradient clipping, empty batch, NaN labels, and zero gradient edge cases. NeuralDBG passed all 15 scenarios (100%) with zero crashes and zero false positives on the healthy baseline.

5.5 Architecture Fuzzer

We built a random valid-model generator spanning 10 layer types. Across 20 randomly generated architectures (seed 42, deterministic), the generator produced zero crashes after fixing spec-generation bugs, and NeuralDBG detected 19/19 manifested bugs. One run injected an asymptomatic fault (a LayerNorm absorbed an fp16 scale perturbation, leaving no detectable failure); we report it as-is rather than forcing a detection. This demonstrates that even “valid” random architectures frequently contain silent bugs that NeuralDBG can detect pre-training.

5.6 Comparison with Existing Tools

We compare NeuralDBG against two real baselines on six canonical failure scenarios (identical architectures, seeds, and batch sizes):

1. `torch.autograd.detect_anomaly()` [1]: PyTorch’s built-in anomaly detection, which traces NaN gradients to specific operations via autograd metadata.
2. Threshold-based monitoring: realistic gradient-norm and loss-spike thresholds simulating what a practitioner would configure in Weights & Biases or TensorBoard.

Table 3: Competitive benchmark results (5 injected bugs + 1 healthy control, identical setup)

Scenario	<code>detect_anomaly</code>	W&B/TB	NeuralDBG	Chains
Exploding gradients	×	✓ (26 alerts)	✓ (19 events)	30
Vanishing gradients	×	✓ (38 alerts)	✓ (15 events)	30
NaN data injection	✓ (1 error)	✓ (10 alerts)	✓ (8 events)	30
Dead neurons	×	✓ (50 alerts)	✓ (7 events)	30
Zero initialization	×	✓ (40 alerts)	✓ (6 events)	30
Detection rate	1/5 (20%)	5/5 (100%)	5/5 (100%)	150
Healthy control (FP gate)	0 FP	0 alerts	0 FP	0

Key findings: (1) `detect_anomaly()` only catches NaN (1/5). It misses exploding, vanishing, dead neurons, and zero initialization—all failures that degrade training without producing NaN. (2)

Detection parity is not the story. Both threshold-based monitoring and NeuralDBG achieve 5/5 detection, but threshold monitoring emits 164 raw alerts with no causal ordering, while NeuralDBG emits 55 structured events organized into 150 causal chains linking root causes to symptoms. The healthy control is reported separately as a false-positive gate (all tools silent, correct behavior) rather than counted as a sixth scenario. (3) Root cause identification is unique to NeuralDBG. On the NaN injection scenario, threshold monitoring reports “loss is NaN” while NeuralDBG identifies the NaN originated in the data pipeline, not the model. (4) The benchmark script (`benchmark_honest.py`) is self-contained and runs in <30 seconds on CPU.

5.7 Out-of-Sample Validation

We tested NeuralDBG on four production-grade architectures never seen during development: torchvision ResNet-18 (11.2M parameters), a custom Vision Transformer (ViT-Tiny), an EfficientNet-B0 implementation, and a Mamba-inspired state-space model, using architecture-agnostic bug injectors. NeuralDBG achieved 24/24 detection (100%) with zero false positives (healthy baselines: 1/2/0/0 events). Earlier runs that reported lower scores were traced to harness bugs (a removed `torch.silu` alias crashing Mamba builds; ResNet-specific injectors acting as no-ops on other architectures), not to detection limits; we document this history in the supplementary audit rather than citing those runs as results.

6 NeuralPrune — Non-Destructive Redundancy Diagnostic

NeuralPrune addresses a complementary problem: identifying parameters that can be removed without affecting model quality. It piggybacks on NeuralDBG’s forward/backward hooks to collect per-layer statistics over a warmup window (default 50 steps). After analysis, it emits a **PruneReport** with estimated redundant parameter counts, memory savings, and confidence-scored recommendations across 7 signal types: DEAD_NEURON, REDUNDANT_WEIGHT, STATIC_WEIGHT, LOW_RANK, QUANTIZABLE, DEAD_EXPERT, and OVERLAPPING_FILTERS. On a test model with deliberately redundant weights, NeuralPrune correctly identified 47.6% of parameters as redundant (8,192/17,226). NeuralPrune is released as a standalone package under the MIT license.

7 Post-Mortems — Reproducing Known PyTorch Bugs

Using NeuralDBG during development, we reproduced and diagnosed ten cataloged PyTorch training failures with causal chains, including `svdvals()` silently swallowing NaN (issue #187759), MPS gradient corruption of $100\times-100K\times$ (#177116), and `varlen_attn()` producing silent NaN with padding (#176793). Two upstream diagnostic test PRs were submitted to PyTorch (#188053, #188923); as of publication, none has been merged. Our process lesson, from maintainer feedback, is to discuss on the issue and secure the *actionable* label before opening a PR—a premature test PR for a refuted premise was withdrawn. Each post-mortem includes the causal chain that NeuralDBG produced, the confirmed root cause, and the lesson for training failure modes.

8 Discussion

8.1 When Does It Work?

NeuralDBG excels at detecting catastrophic failures: exploding gradients (100%), vanishing gradients (100%), divergence (100%), and dead biases (100%). On novel architectures (MoE, Diffusion), it

achieves 100% detection; FlashAttention is partial in-sweep (12/18) while reaching 100% in the Tier 2 setup. These produce large, unmistakable signatures in gradient and activation statistics.

8.2 When Does It Struggle?

Three limitations remain: (1) Quantized models (INT4) introduce gradient noise that partially masks bug signals (83% vs 100% for fp32). (2) GNN tuple inputs—backward hooks do not fully capture gradient flow for modules receiving tuple inputs (88%). (3) An absolute saturation heuristic ($|x| > 0.95$, designed for sigmoid/tanh) misfires on unbounded Linear outputs, masking 7 sweep residuals; fixing it requires an engine change with full re-validation.

8.3 False Positive Reduction

Initial versions exhibited high false positive rates on deep architectures: a healthy ResNet-18 produced 142 anomaly events over 20 steps. Four fixes addressed this: (1) first-encounter baseline events are recorded silently, (2) calibrated distribution shift thresholds with anti-oscillation debounce, (3) per-step gating limiting each layer to one data anomaly event per step, (4) classifier head noise suppression. After these fixes, the healthy baseline produces only 5 events—a 96% reduction—without compromising detection (all 6 failure scenarios remain at 100%).

9 Limitations & Future Work

1. **Out-of-sample validation:** Four architectures (ResNet-18, ViT, EfficientNet, Mamba) achieve 100% detection. Data remains synthetic (CIFAR-shaped random tensors); real-data validation with CIFAR-10/ImageNet is the next priority.
2. **Causal chain quality:** Root cause identification sometimes misattributes when multiple failures occur simultaneously. A linear probe on frozen Qwen2-0.5B embeddings (29/29 holdout) assists diagnosis via learned priors; earlier generative fine-tuning attempts failed (13.9% and 0.71%) and are documented as negative results, not capabilities.
3. **Upstream integration:** Two diagnostic test PRs submitted to PyTorch (#188053, #188923); none merged as of publication. Our process lesson, from maintainer feedback, is to discuss on the issue and secure the *actionable* label before opening a PR. Long-term, a standardized training diagnostic hook API would benefit the entire ecosystem.
4. **Self-evolution:** A 7-step daily pipeline (Scrape→Fuzz→Test→Train→Retrain→Heal→Report) runs verification and discovery; all verified detection gains to date come from human audit of its outputs, so we describe it as a verification loop, not autonomous self-improvement. Multi-day continuous improvement remains to be demonstrated.

10 Conclusion

NeuralDBG demonstrates that causal debugging of deep learning training is feasible and practical. By hooking into PyTorch’s autograd and extracting semantic events, we construct causal chains linking root causes to symptoms across 200 architecture configurations (1,200 evaluations) plus tiered black-swan catalogs. Our detection rates—99.4% on the combinatorial sweep, 96% on Tier 1 black-swans, 94% on Tier 2, 100% on stress tests, 24/24 out-of-sample—show that the approach generalizes beyond standard architectures. NeuralPrune extends the diagnostic paradigm

to model optimization, identifying redundant parameters without weight modification. Ten cataloged PyTorch failures were reproduced and diagnosed, with two upstream diagnostic test PRs submitted (none merged as of publication), and benchmark results showing NeuralDBG equals the detection rate of production monitoring tools while providing 150 causal chains where they provide zero. The system is open-source (MIT), non-invasive (single context manager), and available at <https://github.com/LambdaSection/NeuralDBG>.

References

- [1] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 32, 2019.
- [2] M. Abadi et al. TensorFlow: A System for Large-Scale Machine Learning. *OSDI*, 2016.
- [3] L. Biewald. Experiment Tracking with Weights and Biases. *Weights & Biases Inc.*, 2020.
- [4] D. Nigenda et al. Amazon SageMaker Debugger. *MLSys*, 2022.
- [5] K. Pei et al. DeepXplore: Automated Whitebox Testing of Deep Learning Systems. *SOSP*, 2017.
- [6] A. Odena et al. TensorFuzz: Debugging Neural Networks with Coverage-Guided Fuzzing. *ICML*, 2019.
- [7] E. Schoop et al. UMLAUT: Debugging Deep Learning Programs. *PLDI*, 2022.
- [8] L. Theis et al. Detecting Anomalous Training Runs. *arXiv preprint*, 2017.
- [9] J.A. Jones and M.J. Harrold. Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique. *ASE*, 2005.
- [10] R. Abreu et al. On the Accuracy of Spectrum-based Fault Localization. *TAIC PART*, 2007.
- [11] P. Spirtes et al. *Causation, Prediction, and Search*. MIT Press, 2nd edition, 2000.
- [12] J. Zhang. On the Completeness of Orientation Rules for Causal Discovery. *Artificial Intelligence*, 172(16–17), 2008.
- [13] C.W.J. Granger. Investigating Causal Relations by Econometric Models. *Econometrica*, 37(3), 1969.
- [14] N. Kokhlikyan et al. Captum: A Unified Model Interpretability Library for PyTorch. *arXiv:2009.07896*, 2020.
- [15] M.T. Ribeiro et al. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. *KDD*, 2016.
- [16] S.M. Lundberg and S.I. Lee. A Unified Approach to Interpreting Model Predictions. *NeurIPS*, 2017.
- [17] C. Le Goues et al. Automated Program Repair. *Communications of the ACM*, 62(12), 2019.
- [18] J.O. Kephart and D.M. Chess. The Vision of Autonomic Computing. *IEEE Computer*, 36(1), 2003.

- [19] M. Chen et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.
- [20] H. Pearce et al. Can OpenAI’s Codex Fix Bugs? *APR Workshop*, 2022.
- [21] S. Han et al. Learning Both Weights and Connections for Efficient Neural Networks. *NeurIPS*, 2015.
- [22] P. Molchanov et al. Pruning Convolutional Neural Networks for Resource Efficient Inference. *ICLR*, 2017.
- [23] V. Sanh et al. Movement Pruning: Adaptive Sparsity by Fine-Tuning. *NeurIPS*, 2020.
- [24] T.J. Yang et al. NetAdapt: Platform-Aware Neural Network Adaptation. *ECCV*, 2018.
- [25] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention. *NeurIPS*, 2022.
- [26] R.T.Q. Chen et al. Neural Ordinary Differential Equations. *NeurIPS*, 2018.
- [27] N. Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer. *ICLR*, 2017.
- [28] NeuralDBG. LambdaSection/NeuralDBG. GitHub repository. <https://github.com/LambdaSection/NeuralDBG>, 2026.